Verifiable one-way file transfer across air gaps with screens, paper and serial cables

Taha Bayar — Lumina Digitale, Ankara · ORCID 0009-0009-6247-4985

Preprint, version 1, 30 September 2026. DOI: https://doi.org/10.5281/zenodo.23064776. The figures in §4 are regenerated by the published measurement script; §5–§7 give the measurement behind each number. Contact: https://kionel.com/contact.html

Abstract

Laboratory instrument PCs, industrial workstations and other isolated computers still exchange files through USB storage, a route that security programmes try to close and that leaves no record of what moved. We describe a browser-based system that moves a file from such a machine over channels that are one-way by construction — a screen read by a camera, printed pages read later by any camera, and a serial cable with only the transmit line connected — and that produces a dispatch record signed by the sender and a delivery receipt signed by the receiver, verifiable offline by an open-source tool. The handshake and the two one-way modes are modelled in ProVerif; secrecy, authentication and receipt soundness hold under stated assumptions, and sanity variants that remove each assumption yield the expected attacks. For printed pages, where whole codes are lost to smudges and tears, we show that choosing the spare fountain-coded symbols so that each covers about a third of the source blocks, combined with Gaussian elimination when the peeling decoder stalls, raises recovery of a 30 KB file with 8 of its 48 codes missing from 3% to 91% of trials at about 30% overhead, and of a 150 KB file with 12 codes missing from 4% to 100%. For live camera reading, restricting decoding to the region around the last decoded code reduced delivery time of a 200 KB transfer by about a quarter on a processor slowed to phone speed. We report the limits: the measurements come from one phone and synthetic benchmarks, and the system has had no independent security audit.

1. Problem

Many regulated laboratories keep instrument PCs off the network because their operating systems are no longer patched or because validation forbids change. Their results still have to reach a LIMS. In practice they leave on a USB stick, and the only record of the movement is a form filled in afterwards. Hardware data diodes enforce one-way flow between networks but are priced per link in the thousands to over a hundred thousand dollars, and they are built for streams between networks rather than for a few files a day with a per-file record.

The requirements we set were: nothing installed on the isolated machine (a browser, or one HTML file with a published hash); no path from the receiving side back to the isolated machine; the file's content never reaching a third party; and, per file, evidence that a named file with a given SHA-256 left one device and arrived intact on another, bound to a work order, that an auditor can check without the vendor.

2. System

Channels.

1. *Screen to camera*: encrypted, fountain-coded QR frames shown on the sender; a phone reads them live or from a screen recording. A libcimbar colour-code variant carries more data per image and reads best from a recording.
2. *Printed pages*: the same frames laid out six or twelve to an A4 or Letter page, read later in any order by a phone camera or from photos and scans.
3. *Serial cable*: the frames written over RS-232 or a USB serial adapter via Web Serial, each wrapped with a marker and length so the receiver can resynchronise after corruption; with only TX and ground wired, the link is physically one-way. An unattended mode watches a folder and sends each new or changed file.

Keys and modes. The two-way mode is an X25519 handshake over the optical channel, authenticated by the operator comparing short codes on both screens; it gives forward secrecy. Two one-way modes need no back channel: a 12-character code (60 bits) typed on the sender, from which a key is derived with PBKDF2-SHA-256 (600,000 iterations), and a *receiver card*, a long-term public key of the receiving device installed once on the sender.

Records. The sender signs (Ed25519) a dispatch record over the file's SHA-256, size, time and a context — work order, classification, target system and the operator's declared name. The receiver checks the file against it and signs a delivery receipt that embeds the dispatch record. A receipt can carry an RFC 3161 timestamp; only its fingerprint is sent to the timestamp authority. Receipts on a device are hash-chained. A file can also be *sealed*: only the signed record of its hash leaves, as one code or on paper.

3. Formal analysis

The three protocols are modelled by hand from the implementation in ProVerif 2.05 (`formal/`). Results, including sanity variants in which ProVerif must find an attack:

Model	Variant	Secrecy	Authentication	Receipt soundness
Two-way handshake	codes compared; identity keys leak afterwards	holds (forward secrecy)	holds (injective)	holds
Two-way handshake	codes not compared	attack	attack	holds
One-way, typed code	—	holds while the code is unknown	holds	—
One-way, typed code	offline guessing	possible (expected)	—	—
One-way, receiver card	card installed over a trusted route	holds	holds	—
One-way, receiver card	card sent over the attacker's channel	attack	holds	—
One-way, receiver card	receiver's long-term key leaks later	fails (no forward secrecy)	holds	—

The models are symbolic: they say nothing about implementation bugs, side channels or the strength of the 60-bit code against offline guessing, which is priced by PBKDF2 rather than prevented.

4. Erasure coding for printed pages

A printed code is lost whole or not at all. The frames are an LT code (robust soliton, Luby 2002) made systematic: every source block is printed once, followed by about 30% spare symbols (at least two). We compare two choices of spares and two decoders:

- *random*: the next LT symbols, whose degrees follow the soliton distribution, so most spares combine one or two blocks;
- *dense*: LT seeds selected so that each spare combines at least a third of the source blocks; the receiver treats them as ordinary LT symbols, so the wire format does not change;
- *peel*: the usual belief-propagation (peeling) decoder;
- *elim*: peeling, plus Gaussian elimination over GF(2) on the remaining unknowns (at most 256) when enough equations are pending. Rank is first tested on the bit matrix only; payloads are processed once, after full rank is found.

Each trial removes a uniformly random set of codes, reads the rest in random order and counts a success only if the rebuilt file equals the original.

5 KB file: 6 source blocks, 12 printed codes (6 spare), 300 trials per cell. Share of trials in which the file was recovered exactly:

Spares	Decoder	0 lost	1 lost	2 lost	3 lost	5 lost	8 lost	12 lost
random	peel	100%	100%	98%	91%	53%	0%	0%
random	elim	100%	100%	98%	90%	55%	0%	0%
dense	peel	100%	100%	99%	93%	66%	0%	0%
dense	elim	100%	100%	99%	97%	67%	0%	0%

30 KB file: 36 source blocks, 48 printed codes (12 spare), 300 trials per cell. Share of trials in which the file was recovered exactly:

Spares	Decoder	0 lost	1 lost	2 lost	3 lost	5 lost	8 lost	12 lost
random	peel	100%	96%	79%	61%	20%	3%	0%
random	elim	100%	97%	82%	60%	18%	3%	0%
dense	peel	100%	100%	100%	99%	87%	24%	0%
dense	elim	100%	100%	100%	100%	99%	91%	23%

150 KB file: 179 source blocks, 234 printed codes (55 spare), 100 trials per cell. Share of trials in which the file was recovered exactly:

Spares	Decoder	0 lost	1 lost	2 lost	3 lost	5 lost	8 lost	12 lost
random	peel	100%	88%	77%	66%	45%	12%	4%
random	elim	100%	89%	77%	66%	46%	20%	4%
dense	peel	100%	100%	100%	100%	100%	100%	62%
dense	elim	100%	100%	100%	100%	100%	100%	100%

With 6 source blocks and 6 spares, losing 8 of 12 codes leaves fewer symbols than blocks, so no decoder can succeed; the 5 KB rows at 8 and 12 lost codes are a bound, not a weakness of one method.

The spare selection matters more than the decoder: with random spares, elimination changes almost nothing, because a low-degree spare often does not contain the lost block at all and no decoder can recover information that is not there. With dense spares, elimination adds the rest: at 30 KB and 8 lost codes, peeling alone recovers 24% of trials and peeling plus elimination 91%. The receiver's elimination also helps the camera channel: in one screen-recording test the file was complete after 142 codes instead of 176.

5. Live camera reading

On the receiving phone the cost per frame is dominated by copying and decoding a full camera frame. In a desktop micro-benchmark (zxing-wasm, one version-27 code in a 1920×1080 frame), multi-symbol exhaustive mode took 11.5 ms per frame, single-symbol fast mode 9.1 ms, and the same frame at half resolution 3.8 ms, all decoding 15 of 15. After a code is found, the receiver now reads only the union of the codes' bounding boxes enlarged by half their size on each side, with a full frame every tenth frame or whenever the region yields nothing. In an end-to-end test with a fake camera stream (1280×720, the code filling about 60% of the height) and Chrome's CPU throttled twelvefold, a 200 KB transfer completed in 35 s before and 26–27 s after (two runs each). The gain depends on how much of the frame the code fills; on a real phone we have not yet measured it.

6. The serial link

At 8N1 a byte costs ten bits, so 115,200 baud carries about 11.5 KB/s and 921,600 baud about 92 KB/s before framing (about 3%) and spare symbols (10%). Without a back channel the sender cannot know when the receiver has finished; it sends one full systematic pass plus spares and offers to continue the same session. In the unattended mode each file is a new session; the receiver remembers delivered sessions so that late spare frames are not mistaken for a new transfer. A file sent while the receiver is off is lost, by design; the sender's log lists what was sent.

7. Observations on a real phone

- Screen to camera, 117 KB: 10–24 s read live, about 7 s from a screen recording.
- Printed pages displayed on a monitor and scrolled past the camera, 93 KB: 1 min 45 s; 144 of 1,150 decoded codes were new, the rest repeats of the page in view, so scrolling speed rather than decoding set the time. The file was complete before the last page.
- A screen recording sent through a messaging app arrived at 478×850 and could not be decoded; the system now detects the reduced resolution and says why.

8. Limitations

All phone numbers come from one device. The throttled-CPU benchmark is a proxy for a phone, not a measurement of one. The formal models are hand-written, not extracted. The system inspects no file content and has had no independent security audit. The code is closed source except the verifier (Apache-2.0).

9. Related work

Animated-QR transfer with fountain codes appears in TXQR, qr-send (Wirehair), and several recent browser tools; libcimbar provides high-density colour codes. Paper backup tools include paperkey for GnuPG keys, qr-backup and PaperCrypt. Raptor and RaptorQ codes (RFC 6330) use inactivation decoding to approach MDS behaviour; our spare selection is a lighter alternative that keeps a plain LT receiver. Hardware data diodes, including newer low-cost designs, enforce one-way flow between networks. We are not aware of published work that combines such one-way channels with signed per-file dispatch and delivery records verifiable offline; we would welcome pointers.

10. Availability

Verifier: Apache-2.0. ProVerif models and runner: `formal/`, also downloadable from the website. Measurement script for §4: `research/paper-loss.test.ts`; data in `research/data/`.

References

- M. Luby. LT codes. FOCS 2002.
- A. Shokrollahi. Raptor codes. IEEE Transactions on Information Theory 52(6), 2006.
- RFC 6330, RaptorQ Forward Error Correction Scheme for Object Delivery, 2011.
- B. Blanchet. ProVerif. https://bblanche.gitlabpages.inria.fr/proverif/
- RFC 3161, Internet X.509 PKI Time-Stamp Protocol, 2001.
- libcimbar. https://github.com/sz3/libcimbar
- qr-backup. https://github.com/za3k/qr-backup
- paperkey. https://www.jabberwocky.com/software/paperkey/